

PART 1 - Problem 1

Tag List: [STANDARD ASSESSMENT] [PARTIAL CREDIT] [SYMBOLIC MATH TOOLBOX] [CONTROL TOOLBOX]

Consider the **linear system** described by the **state equations**:

$$\begin{cases} \dot{x}(t) &= A x(t) + B u(t) \\ y(t) &= C x(t) \end{cases}, \quad A = \begin{bmatrix} 0 & -4 \\ 1 & p_1 \end{bmatrix}, \quad B = \begin{bmatrix} p_2 \\ p_3 \end{bmatrix}, \quad C = [0 \quad 1]$$

where $p_1, p_2, p_3 \in \mathbb{R}$ are generic parameters. Assume as initial state $x(0)$ the following

$$\begin{bmatrix} x_1(0) \\ x_2(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Assignment

Your code shall solve the following three problems regarding some features of the dynamic system described by the above equations.

P1: Permanent Oscillations in the System's Step Response.

- Determine the value of p_1 for which the **system's step response** has **permanent oscillations**.
- **Hint:** The initial state of the system is zero. It is, therefore, convenient to determine the system's transfer function (to compute the system's 'forced response' to any causal input) and then analyse it (refer to **Part 4** of the course material). Does any feature of the transfer function depend on the parameters? What feature? How do the parameters influence the temporal behaviour of the system's forced response?
- What **period** do these oscillations have?
- Your code should **determine** the specific value of the parameter p_1 and **assign** it to the numeric variable **P0_p1**.
- Finally, **compute** the oscillations' period and assign the value to the numeric variable **P0_T**.

P2: Steady-State Value and Overshoot of the System's Step Response

- Assume $p_3 = 0$.
- Determine the **parameters** p_1 and p_2 for which the system's step response tends to the **regime value** $y_\infty = 100$ with **maximum percentage overshoot** $\Delta\% = 25$.
- **Hint 1:** If you apply a stepwise constant input to an asymptotically stable LTI system, then ... (refer to **Part 3** of the course material).

- **Hint 2:** What are the characteristic parameters of the step response? How to evaluate such parameters? Refer to **Part 7** of the course material. Do p_1 , p_2 , and p_3 have an impact on the characteristic parameters of the step response?
- Your code should **determine** the specific values of the parameters p_1 , p_2 and **assign** those values to the numeric variables **R0_p1** and **R0_p2**.

Remark: You do not have to solve problem P1 to solve problem P2.

P3: Settling Time of the System's Step Response

- Assume $p_3 = 0$. Moreover, use for p_1 and p_2 the previously determined values **R0_p1** and **R0_p2**.
- **Evaluate** the **settling time** of the system's step response using the appropriate approximate settling time formulation.
- **Hint:** What are the characteristic parameters of the step response? How to evaluate such parameters? Refer to **Part 7** of the course material. Do p_1 , p_2 , and p_3 have an impact on the characteristic parameters of the step response?
- **Assign** the computed settling time value to the numeric variable **ST_T**.
- **Calculate** the settling time at 1% of the LTI system's step response, using the Matlab function **stepinfo()** appropriately. **Store** the obtained settling time in the numeric variable **ST_SI_T**.
- **Hint:** Detailed help on how to use the **stepinfo()** function is available [here](#).

Solution

Solution to the Problem P1

The system is second-order. The step response exhibits permanent oscillations if the system's poles are pure imaginary complex values. Thus, we have to impose zero damping: the characteristic polynomial is

$$\varphi(s) = \det(sI - A) = \det \begin{bmatrix} s & 4 \\ -1 & s - p_1 \end{bmatrix} = s(s - p_1) + 4 = s^2 - p_1s + 4$$

If p_1 is zero, the polynomial has imaginary roots. In particular, the roots are $s_{1,2} = \pm j2$. These **imaginary poles** are **responsible** for the **permanent oscillations** in the unit step response.

[illegible]

```
% -- the effective eigenvalues
```

```
pA0 = subs(pA, p1, P0_p1)
```

```
pA0 =  $s^2 + 4$ 
```

```
P0_eig = solve(pA0)
```

```
P0_eig =
```

```
 $\begin{pmatrix} -2i \\ 2i \end{pmatrix}$ 
```

```
% -- evaluation of the period of the permanent oscillations
```

```
P0_time_period = (2*pi)/imag(P0_eig(2))
```

```
P0_time_period =  $\pi$ 
```

```
P0_T = double(P0_time_period) % <<<<<<<<----- Second partial solution of P1
```

```
P0_T =  
3.1416
```

Solution to the Problem P2

Let's calculate the expression of the system transfer function (as symbolic expression)

```
p3 = sym(0); % P2 initial assumption
```

```
% -- the matrices B and C ---
```

```
syms p2 % the remaining symbolic parameter
```

```
B = [p2; p3];
```

```
C = sym([0 1]);
```

```
% -- the matrices B and C ---
```

```
% --- the transfer function ---
```

```
sI_A = s*eye(size(A))-A;
```

```
sI_Ainv = inv(sI_A);
```

```
Gs = C*sI_Ainv*B
```

```
Gs =
```

```
 $\frac{p_2}{s^2 - p_1 s + 4}$ 
```

```
% --- the transfer function ---
```

```
% --- asymptotic stability requirement
```

```
assume(p1<0)
```

```
% --- asymptotic stability requirement
```

```
% --- static gain
```

```
gainG = subs(Gs,s,0)
```

```
gainG =
```

```
 $\frac{p_2}{4}$ 
```

```
% --- solution for p2 <<<<-----
Yinfy = sym(100); % the assigned steady-state value of the system's step response
U = sym(1); % the input step size
```

```
eq_p2 = gainG*U==Yinfy;
p2_sol = solve(eq_p2, p2) % <<<<----- the symbolic solution for p2
```

```
p2_sol = 400
```

```
R0_p2 = double(p2_sol) % <<<<<<<<<----- the required solution for p2
```

```
R0_p2 =
400
```

```
% --- solution for p2 <<<<-----
```

The value of p_1 can be derived (refer to the Part 7 of the class material, pp. 21-29; in particular refer to Part 7, p. 27) by recalling that the maximum overshoot of the step response depends only on the damping:

$$\Delta\% = 100 e^{-\frac{\pi \xi}{\sqrt{1-\xi^2}}} = 25 \implies e^{-\frac{\pi \xi}{\sqrt{1-\xi^2}}} = 0.25 \implies \pi^2 \xi^2 = [\ln 0.25]^2 (1 - \xi^2)$$

Thus

$$\bar{\xi} = \frac{\ln 0.25}{\sqrt{\pi^2 + (\ln 0.25)^2}}$$

```
syms xi % <-- define the damping factor
assume(xi<=1); assumeAlso(xi>=0);
assumptions(xi)
```

```
ans = (0 ≤ ξ ≤ 1)
```

```
xi_eq = (pi*xi)^2==(log(0.25))^2*(1-xi^2); % <-- the assigned max overshoot
xi_sol = vpa(solve(xi_eq, xi),6)% <-- the corresponding damping factor
```

```
xi_sol = 0.403713
```

Finally, let's compare the polynomial at the denominator in the transfer function

$$G(s) = \frac{p_2}{s^2 - p_1 s + 4} \implies s^2 - p_1 s + 4$$

with the classic expression of a second-order polynomial with complex conjugate roots having natural angular frequency ω_n and damping factor ξ (refer to the Part 7 of the class material, p. 26)

$$s^2 - p_1 s + 4 \iff s^2 + 2\xi\omega_n s + \omega_n^2$$

Given the natural frequency $\omega_n = 2$ rad/s (by comparison of the zero-degree terms), we obtain the relationship between the parameter p_1 and the damping factor ξ :

$$\xi = -\frac{p_1}{4} \implies p_1 = -4\xi$$

Thus

```
R0_p1 = -4*double(xi_sol) % <<<<<<<<<<----- the required solution for p1
```

```
R0_p1 =
-1.6149
```

Let's verify: using the just computed p_1 value, the poles of the transfer function $G(s)$ are

```
polesG = roots([1 -R0_p1 4])
```

```
polesG = 2x1 complex
-0.8074 + 1.8298i
-0.8074 - 1.8298i
```

Referring to the Part 7 of the course material (refer to Part 7, p. 21 and 25), such poles should have the following natural angular frequency and damping factor:

poles:	$-\sigma \pm j \omega$
angular natural frequency ω_n :	$\omega_n^2 = \sigma^2 + \omega^2$
damping factor ξ :	$\omega_n \xi = \sigma$

Thus:

```
sigmaP = abs(real(polesG(1)))
```

```
sigmaP =
0.8074
```

```
omegaP = abs(imag(polesG(1)))
```

```
omegaP =
1.8298
```

```
omegaN_p = sqrt(sigmaP.^2+omegaP.^2)
```

```
omegaN_p =
2
```

```
dampFactP = sigmaP/omegaN_p
```

```
dampFactP =
0.4037
```

Finally, the corresponding maximum overshoot of the step response holds

```
maxS = 100 * exp( -(pi*dampFactP)/sqrt(1-dampFactP.^2)) )
```

```
maxS =
25.0000
```

As expected.

Solution to the Problem P3

Summing up, we are considering the linear system described by the state equations:

$$\begin{cases} \dot{x}(t) &= A x(t) + B u(t) \\ y(t) &= C x(t) \end{cases}, \quad A = \begin{bmatrix} 0 & -4 \\ 1 & p_1 \end{bmatrix}, \quad B = \begin{bmatrix} p_2 \\ p_3 \end{bmatrix}, \quad C = [0 \quad 1]$$

where $p_1, p_2, p_3 \in \mathbb{R}$ are assuming the values

$$p_1 = \text{RO_p1}, \quad p_2 = \text{RO_p2}, \quad p_3 = 0$$

Moreover, the initial system state $x(0)$ is the following

$$\begin{bmatrix} x_1(0) \\ x_2(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The settling time of the system's step response depends on the poles of the transfer function, according to the formulation (refer to the Part 7 of the course material, slides 21-27)

$$G(s) = \frac{q}{s^2 + 2\xi\omega_n s + \omega_n^2} = \frac{p_2}{s^2 - p_1 s + 4} \implies t_{s\,1\%} \approx \frac{4.6}{\xi \omega_n} = \frac{4.6}{\frac{(-p_1)}{2}} = \frac{9.2}{(-p_1)}$$

Thus

```
ST_T = 9.2/(-RO_p1)    % <<<<----- settling time according the classic formulation
```

```
ST_T =  
5.6971
```

To estimate the settling time of the step response using the Control System Toolbox function **stepinfo()**, we need to create the system description as an LTI object in Matlab:

```
% --- the matrices A, B, C and D as numerical matrices  
matA = double(subs(A, p1, RO_p1));  
matB = double(subs(B, [p2, p3], [RO_p2, 0]));  
matC = double(C);  
matD = zeros(1);  
% --- the matrices A, B, C and D as numerical matrices  
  
% --- let's define the LTI object ---  
LTIsys = ss(matA, matB, matC, matD);  
% --- let's define the LTI object ---  
  
% -- let's retrieve the info on the step response --  
% we request the evaluation of the settling time with 0.01 as threshold  
StepOutputInfo = stepinfo(LTIsys,"SettlingTimeThreshold", 0.01);  
  
% the settling time is  
ST_SI_T = StepOutputInfo.SettlingTime
```

```
ST_SI_T =  
5.6515
```

Remark: the resulting settling time values ST_T and ST_{SI_T} are very close to each other, but they are not identical. The first value is obtained through a theoretical but approximated formulation, and the latter results from evaluating the numerical step response.

